

計算機科学実験 1

レポート課題 5

学籍番号 6324029

氏名 北澤泰樹

2025 年 8 月 8 日

問題 1：SProlog の生成規則と抽象構文木の対応をしながら、SProlog 処理系を完成させなさい。

1 ソースコード

```
1 推論
2 (**)
3 module Evaluator = struct
4 type ast =
5   Atom of string
6   | Var  of string
7   | App  of string * ast list
8
9 module P = Printf
10
11 (*let rec print_ast ast = match ast with*)
12
13
14 let rec print_ast ast = match ast with
15 | (App (s, hd :: tl)) ->
16   (P.printf "App(\"%s\",[" s;
17    print_ast hd;
18    List.iter (fun x -> (print_string ";" ; print_ast x)) tl;
19    print_string "])")
20 | (App (s, [])) ->
21   P.printf "App(\"%s\",[])" s
22 | (Atom s) -> P.printf "Atom \"%s\"" s
23 | (Var s) -> P.printf "Var \"%s\"" s
24
25 let print_ast_list lst =
26   match lst with
27 | (hd :: tl) ->
28   (print_string "[";
29    print_ast hd;
30    List.iter (fun x -> print_string ";" ; print_ast x) tl;
31    print_string "]")
32 | [] -> print_string "[]"
33
34 let sub name term =
35 let rec mapVar ast =
36   match ast with
37   (Atom x) -> Atom(x)
38   | (Var n)   -> if n = name then term else Var n
39   | (App (n, terms)) -> App (n, List.map mapVar terms)
40 in mapVar
41
42 let mgu (a, b) =
43   let rec ut (one,another,unifier) = match (one,another) with
44     ([],[]) -> (true,unifier)
```

```

45 | (term::t1, Var(name)::t2) ->
46   let r= fun x->sub name term (unifier x) in
47     ut(List.map r t1, List.map r t2,r)
48   |(Var(name)::t1,term::t2)->
49   let r= fun x-> sub name term (unifier x) in
50   ut(List.map r t1, List.map r t2,r)
51 |(Atom(n)::t1,Atom(m)::t2)->
52 if n = m then ut(t1,t2,unifier)else(false,unifier)
53 |(App(n1,xt1)::t1,App(n2,xt2)::t2)->
54 if n1=n2 && List.length xt1 = List.length xt2 then
55   ut(xt1@t1,xt2@t2,unifier)
56   else(false,unifier)
57 |(_,_-> (false,unifier)
58   in
59   ut ([a], [b], (fun x -> x))
60
61 (* ----- 変数の重複回避のための rename ----- *)
62 let rename ver term =
63 let rec mapVar ast = match ast with
64   (Atom x) -> Atom(x)
65   | (Var n) -> Var (n ^ "#" ^ ver)
66   | (App (n, terms)) -> App (n, List.map mapVar terms)
67 in mapVar term
68
69 (* ----- 深さ優先の解探索 ----- *)
70 exception Compiler_error
71
72 (* succeed: 最終的に得たい「結果の組み立て方」を渡す想定らしかったのでとりあえず恒等写像で置いておく
73   *)
74 let succeed query = (print_ast query; true)
75
76 let rec solve (program, question, result, depth) = match question with
77   []-> succeed result
78   |goal::goals->
79   let onestep _ clause = match List.map (rename (string_of_int depth)) clause
80     with
81     []->raise Compiler_error
82     |head::conds->
83     let (unifiable,unifier) = mgu (head,goal) in
84     if unifiable then
85       solve (program,List.map unifier(conds@goals),
86         unifier result,depth+1)
87     else true
88 in List.fold_left onestep true program
89 let eval (program,question) = solve(program,[question],question,1)
90 end字句回析
91
92 (**)
93 module Lexer = struct

```

```

93 module P = Printf
94
95 type token =
96   | CID of string
97   | VID of string
98   | NUM of string
99   | TO
100  | IS
101  | QUIT
102  | OPEN
103  | EOF
104  | ONE of char
105
106 exception End_of_system
107
108 let line = ref 1
109 let _ISTREAM = ref stdin
110 let ch      = ref []
111
112 let get_line_num () = !line
113
114 let read () =
115   match !ch with
116   | h :: rest -> ch := rest; h
117   | []        -> input_char !_ISTREAM
118
119 let unread c = ch := c :: !ch
120
121 let lookahead () =
122   try
123     let c = read () in
124     unread c; c
125   with End_of_file ->
126     '$'
127
128 let rec integer i =
129   let c = lookahead () in
130   if c >= '0' && c <= '9' then
131     integer (i ^ Char.escaped (read ()))
132   else
133     i
134
135 and identifier id =
136   let c = lookahead () in
137   if
138     (c >= 'a' && c <= 'z') ||
139     (c >= 'A' && c <= 'Z') ||
140     (c >= '0' && c <= '9') ||
141     c = '_'

```

```

142     then
143         identifier (id ~ Char.escaped (read ()))
144     else
145         id
146
147 and native_token () =
148     let c = lookahead () in
149     if c >= 'a' && c <= 'z' then begin
150         let id = identifier "" in
151         match id with
152         | "is"    -> IS
153         | "quit" -> QUIT
154         | "open" -> OPEN
155         | _      -> CID id
156     end
157     else if c >= 'A' && c <= 'Z' then
158         let id = identifier "" in
159         VID id
160     else if c >= '0' && c <= '9' then
161         NUM (integer "")
162     else if c = ':' then begin
163         ignore (read ());
164         if lookahead () = '-' then (ignore (read ()); TO)
165         else ONE ':'
166     end
167     else
168         ONE (read ())
169
170 and gettoken () =
171     try
172         match native_token () with
173         | ONE '\n' -> line := !line + 1; gettoken ()
174         | ONE ' ' | ONE '\t' -> gettoken ()
175         | tk      -> tk
176     with End_of_file -> EOF
177
178 let print_token = function
179     | CID i    -> P.printf "CID(%s)"  i
180     | VID i    -> P.printf "VID(%s)"  i
181     | NUM i    -> P.printf "NUM(%s)"  i
182     | TO      -> P.printf ":-"
183     | QUIT    -> P.printf "quit"
184     | OPEN    -> P.printf "open"
185     | IS      -> P.printf "is"
186     | EOF     -> P.printf "eof"
187     | ONE c   -> P.printf "ONE(%c)"  c
188
189 let rec run () =
190     let rlt = gettoken () in

```

```

191     match rlt with
192     | EOF -> raise End_of_system
193     | _      ->
194         print_token rlt;
195         P.printf "\n%!";
196         run ()
197 end
198
199 module Parser = struct
200     module L = Lexer
201     module E = Evaluator
202     let prog = ref [[E.Var ""]]
203     let tok = ref (L.ONE ' ')
204
205     let getToken () = L.gettoken ()
206     let advance () = tok := getToken ()
207
208     exception Syntax_error
209     let error () = raise Syntax_error
210
211     let check t =
212         match !tok with
213         | L.CID _ -> if (t=(L.CID "")) then () else error()
214         | L.VID _ -> if (t=(L.VID "")) then () else error()
215         | L.NUM _ -> if (t=(L.NUM "")) then () else error()
216         | tk -> if (tk=t) then () else error()
217
218     let eat t = check t; advance ()
219
220     let rec clauses () =
221         match !tok with
222         | L.EOF -> []
223         | _ -> let c = clause() in c :: clauses()
224
225     and clause () =
226         match !tok with
227         | L.ONE '(' -> let c = term() in eat(L.ONE '.');[c]
228         | _ -> let p = predicate () in let o = p :: to_opt () in eat (L.ONE '.');
229             o
230
231     and to_opt () =
232         match !tok with
233         | L.TO -> eat (L.TO); terms ()
234         | _ -> []
235
236     and command () =
237         match !tok with
238         | L.QUIT -> exit 0
239         | L.OPEN ->

```

```

239         ( eat (L.OPEN);
240         match !tok with
241         | L.CID s ->
242             (eat (L.CID "");
243             check (L.ONE ' ');
244             L._ISTREAM := open_in (s ^ ".pl");
245             L.line := 1;
246             advance (); prog := clauses ();
247             close_in (!L._ISTREAM))
248         | _ -> error ()
249 (* | L.ONE '(' | L.VID _ | L.CID _ | L.NUM _ ->
250     (terms (); check(L.ONE ' '))
251     | _ -> error (**)
252     | _ -> let t = term() in (check(L.ONE ' '); let _ = E.eval(!prog, t) in ())
253
254 and term () =
255     match !tok with
256     | L.ONE '(' -> (eat(L.ONE '('); let t = term () in eat (L.ONE ')');t) (*は
257         () に囲われたもののterm*)
258     (*| L.VID s -> (eat(L.VID ""); eat (L.IS); arithmexp ())*
259     | _ -> predicate ()
260
261 and terms () = let t = term () in t :: terms'()
262 and terms' () =
263     match !tok with
264     | L.ONE ',' -> eat (L.ONE ','); let t1 = term () in t1 :: terms' ()
265     | _ -> []
266
267 and predicate () =
268     match !tok with
269     | L.CID s ->
270         eat (L.CID "");
271         eat (L.ONE '(');
272         let a = args () in
273         eat (L.ONE ')') ; E.App(s,a) モジュールの外を使うとき (**)
274     | _ -> error ()
275
276 and args () = let e = expr () in e :: args'()
277 and args' () =
278     match !tok with
279     | L.ONE ',' -> eat (L.ONE ',');let e = expr () in e :: args' ()
280     | _ -> []
281
282 and expr () =
283     match !tok with
284     | L.ONE '(' -> eat (L.ONE '(');let e = expr () in eat (L.ONE ')') ; e ここ変え
285         た (**)
286     | L.ONE '[' -> eat (L.ONE '[');let l = list () in eat (L.ONE ']') ; l ここも
287         (**)
288     | L.CID s -> eat (L.CID ""); tail_opt s

```

```

286 | L.VID s -> eat (L.VID ""); E.Var s
287 | L.NUM n -> eat (L.NUM ""); E.Atom n
288 | _ -> error ()
289
290 and tail_opt s =
291   match !tok with
292   | L.ONE '(' -> eat (L.ONE '('); let a = args () in eat (L.ONE ')') ; E.App (
      s,a)
293   | _ -> E.Atom s
294
295 and list () =
296   match !tok with
297   | L.ONE ']' -> E.Atom "nil"
298   | _ -> let e = expr () in let l = list_opt () in E.App ("cons",[e;l])
299
300 and list_opt () =
301   match !tok with
302   | L.ONE '|' -> eat (L.ONE '|'); id ()
303   | L.ONE ',' -> eat (L.ONE ','); list ()
304   | _ -> E.Atom "nil"
305
306 and id () =
307   match !tok with
308   | L.CID s -> eat(L.CID "");E.Atom s
309   | L.VID s -> eat(L.VID "");E.Var s
310   | L.NUM n -> eat(L.NUM "");E.Atom n
311   | _ -> error ()
312
313 (* and arithmexp () =
314   arithmterm ();
315   arithmexp' ()
316
317 and arithmexp' () =
318   match !tok with
319   | L.ONE '+' -> eat (L.ONE '+'); arithmterm (); arithmexp' ()
320   | L.ONE '-' -> eat (L.ONE '-'); arithmterm (); arithmexp' ()
321   | _ -> ()
322
323 and arithmterm () =
324   arithmfactor ();
325   arithmterm' ()
326
327 and arithmterm' () =
328   match !tok with
329   | L.ONE '*' -> eat (L.ONE '*'); arithmfactor (); arithmterm' ()
330   | L.ONE '/' -> eat (L.ONE '/'); arithmfactor (); arithmterm' ()
331   | _ -> ()
332
333 and arithmfactor () =

```



```

334     match !tok with
335     | L.ONE '(' -> eat (L.ONE '('); arithmexp (); eat (L.ONE ')')
336     | L.ONE '-' -> eat (L.ONE '-'); arithmexp ()
337     | L.ONE '[' -> eat (L.ONE '['); list (); eat (L.ONE ']')
338     | L.CID s -> eat (L.CID s); tail_opt ()
339     | L.VID s -> eat (L.VID s)
340     | L.NUM n -> eat (L.NUM n)
341     | _ -> ()
342     *)
343 end
344
345 let rec run () =
346     print_string "?-";
347     while true do
348         flush stdout;
349         Lexer._ISTREAM := stdin;
350         try
351             Parser.advance ();
352             Parser.command ();
353             print_string "\n?- "
354         with
355         | Parser.Syntax_error ->
356             let line_num = Lexer.get_line_num () in
357             Printf.eprintf "Syntax Error at line %d (token: " line_num;
358             Lexer.print_token !Parser.tok;
359             Printf.eprintf ").\n%!";
360             try
361                 while (Lexer.read ()) <> '\n' do () done;
362                 Lexer.line := !Lexer.line + 1;
363                 print_string "?- "
364             with End_of_file ->
365                 print_string "\n"; exit 0
366         | Lexer.End_of_system ->
367             print_string "\n"; exit 0
368         (*| End_of_file ->
369             print_string "\n"; exit 0*)
370     done

```

fbox 課題 2:isono プログラムを入力し、いくつかの質問について、振舞を確認しなさい。

```
module Evaluator :
sig
  type ast = Atom of string | Var of string | App of string * ast list
  module P = Printf
  val print_ast : ast -> unit
  val print_ast_list : ast list -> unit
  val sub : string -> ast -> ast -> ast
  val mgu : ast * ast -> bool * (ast -> ast)
  val rename : string -> ast -> ast
  exception Compiler_error
  val succeed : ast -> bool
  val solve : ast list list * ast list * ast * int -> bool
  val eval : ast list list * ast -> bool
end
module Lexer :
sig
  module P = Printf
  type token =
    | CID of string
    | VID of string
    | NUM of string
    | TO
    | IS
    | QUIT
    | OPEN
    | EOF
    | ONE of char
  exception End_of_system
  val line : int ref
  val _ISTREAM : in_channel ref
  val ch : char list ref
  val get_line_num : unit -> int
  val read : unit -> char
  val unread : char -> unit
  val lookahead : unit -> char
```

```

    val lookahead : unit -> char
    val integer : string -> string
    val identifier : string -> string
    val native_token : unit -> token
    val gettoken : unit -> token
    val print_token : token -> unit
    val run : unit -> 'a
  end
module Parser :
  sig
    module L = Lexer
    module E = Evaluator
    val prog : E.ast list list ref
    val tok : L.token ref
    val getToken : unit -> L.token
    val advance : unit -> unit
    exception Syntax_error
    val error : unit -> 'a
    val check : L.token -> unit
    val eat : L.token -> unit
    val clauses : unit -> E.ast list list
    val clause : unit -> E.ast list
    val to_opt : unit -> E.ast list
    val command : unit -> unit
    val term : unit -> E.ast
    val terms : unit -> E.ast list
    val terms' : unit -> E.ast list
    val predicate : unit -> E.ast
    val args : unit -> E.ast list
    val args' : unit -> E.ast list
    val expr : unit -> E.ast
    val tail_opt : string -> E.ast
    val list : unit -> E.ast
    val list_opt : unit -> E.ast
    val id : unit -> E.ast
  end
val run : unit -> unit = <fun>

```

図1 実行結果

```

# run();
?-open isono.
?-father(X,Y).
App("father",[Atom "masao",Atom "tara"])App("father",[Atom "namihei",Atom "sazae"])App("father",[Atom "namihei",Atom "katsuo"])App("father",[Atom "namihei",Atom "wakame"])
?-male(X,Y).
?-male(X).
App("male",[Atom "namihei"])App("male",[Atom "katsuo"])App("male",[Atom "tara"])App("male",[Atom "masao"])
?-grandmother(X,Y).
App("grandmother",[Atom "fune",Atom "tara"])

```

図2 振舞の確認

- 2 問題 3: 第 9 章で示した関数 `succeed` では, インスタンス化した質問の構文木を印字するようになっている. これを, SProlog のソース言語の表現で印字するようにしなさい

3 変えた箇所

```
1 let rec print_ast ast = match ast with
2 (App(s, hd::tl)) -> (P.printf "%s(" s;
3 List.iter (fun x -> (print_ast x; print_string ",")) tl; print_ast hd;
4 print_string ")")
5 | (App(s, [])) -> P.printf "%s" s
6 | (Atom s) -> P.printf "%s" s
7 | (Var s) -> P.printf "%s" s
8 let print_ast_list lst = match lst with
9 (hd::tl) -> (print_string "["; print_ast hd;
10 List.iter (fun x -> (print_string ";";
11 print_ast x)) tl; print_string "]")
12 | [] -> print_string "[]"
```

```
1 let succeed query = (print_ast query; P.printf ".\n"; true)
```

4 実行結果

```
val fun : unit -> unit = <fun>  
# run();;  
?-open isono.  
  
?- father(namihei,X).  
father(sazae,namihei).  
father(katsuo,namihei).  
father(wakame,namihei).  
  
?- mother(fune,X).  
mother(sazae,fune).  
mother(katsuo,fune).  
mother(wakame,fune).  
mother(wakame,fune).  
  
?- grandmother(X,Y).  
grandmother(tara,fune).  
  
?- 
```

図3 実行結果

- 5 問題 4: 通常の Prolog は, 質問が真であったとき, インスタンス化した質問を印字するのではなく, 質問に含まれる変数ごとに, 対応する項を印字する.SProlog の処理系もそのように拡張せよ.

6 ソースコードの変更箇所

```
1 行目
2 (*92*)
3 let rec succeed query l = match (query,l) with
4 (h :: tq, Var s :: t) -> P.printf "%s=" s ; print_ast h;P.printf "\n" ; succeed
   tq t
5 |(_ , _) -> true行目
6
7
8 (*112*)
9 let rec solve (program, question, l, result, depth) = match question with
10 [] -> succeed result l
11 | goal::goals ->
12 let onestep _ clause =
13 match List.map (rename (string_of_int depth)) clause with
14 [] -> raise Compiler_error
15 | head::conds ->
16 let (unifiable, unifier) = mgu(head,goal) in
17 if unifiable then
18 solve (program, List.map unifier (conds@goals), l ,List.map unifier result,
   depth+1)
19 else true
20 in List.fold_left onestep true program
```

7 実行結果

```
# run();;
?-open isono.

?- male(X).
X=namihei
X=katsuo
X=tara
X=masuo

?- female(X).
X=fune
X=wakame
X=sazae

?- father(X,Y).
X=masuo
Y=tara
X=namihei
Y=sazae
X=namihei
Y=katsuo
X=namihei
Y=wakame

?-
```

図 4 実行結果

- 8 問題 5: 通常の Prolog は、質問が真である単一化代入が得られるたびに、推論を継続するか終了するかを指示することができる, SProlog の処理系も, succeed が呼ばれるたびに, 実行を中断し, [;] を打ち込むと継続, [.] を打ち込むと, それ以降の処理を回避するように拡張しなさい.

9 ソースコードの変更箇所

```
1 行目
2 (*97*)
3 let rec succeed query l = match (query,l) with
4 (h :: tq, Var s :: t) -> P.printf "%s=" s ; print_ast h;P.printf "\n" ; succeed
   tq t
5 |(_ , _) -> print_string "Yes." ;P.printf "\n"; true行目
6
7
8 (*141*)
9 let eval (program, question) = let l = List.concat(List.map extract question)
   in if
10 (solve(program, question, l, l, 1)) then print_string "No.\n" else ()
11 end
```

10 実行結果


```
# run();;
?-open isono.

?- father(namihei,X).
X=sazae
Yes.
X=katsuo
Yes.
X=wakame
Yes.
No.

?- female(X).
X=fune
Yes.
X=wakame
Yes.
X=sazae
Yes.
No.

?- male(X).
X=namihei
Yes.
X=katsuo
Yes.
X=tara
Yes.
X=masuo
Yes.
No.

?-
```

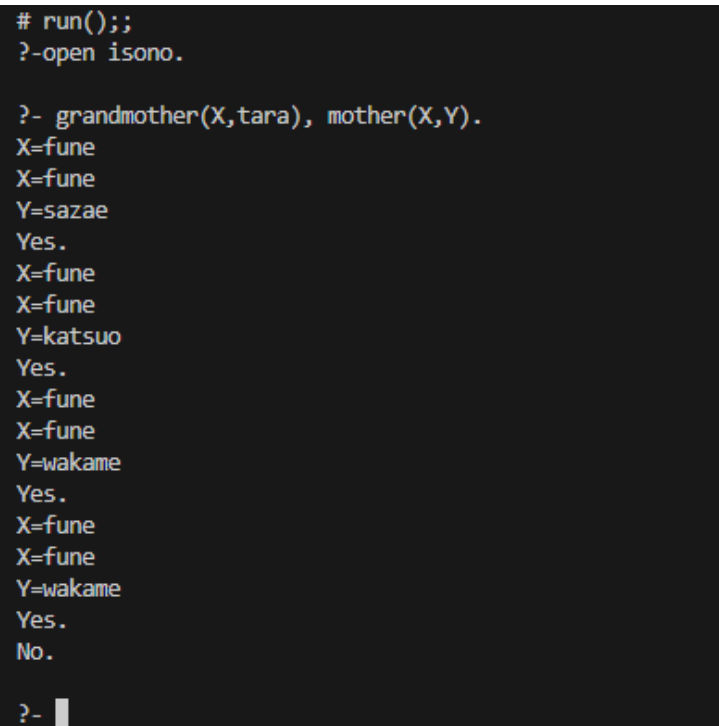
図 5 実行結果

問題 6: [,] で並べた複数質問を受け付けられるように拡張せよ.

11 ソースコードの変更箇所

```
1 And command() = match !tok with
2 L.QUIT -> exit 0
3 | L.OPEN -> (eat(L.OPEN);
4 match !tok with
5 L.CID s -> (eat(L.CID ""); check (L.ONE ' ');
6 L._ISTREAM := open_in (s^".pl"); advance();
7 prog := clauses(); close_in (!L._ISTREAM))
8 | _ -> error()
9 | _ -> let t = terms() in
10 (check(L.ONE ' '); let _ = E.eval(!prog, t) in ())
```

12 実行結果



```
# run();;
?-open isono.

?- grandmother(X,tara), mother(X,Y).
X=fune
X=fune
Y=sazae
Yes.
X=fune
X=fune
Y=katsuo
Yes.
X=fune
X=fune
Y=wakame
Yes.
X=fune
X=fune
Y=wakame
Yes.
No.

?-
```

図 6 実行結果

問題 7: 「VID IS arithmexp」の構文で, 変数への代入を行えるように拡張しなさい.

13 ソースコードの変更箇所

```
1 行目
2  (*258*)
3  let initprog = [E.App("is", [E.Var "X"; E.Var "X"])]行目
4  (*260*)
5  let prog = ref [initprog]行目
6  (*416*)
7  and command() = match !tok with
8  L.QUIT -> exit 0
9  | L.OPEN -> (eat (L.OPEN));
10 match !tok with
11 L.CID s -> (eat (L.CID ""); check (L.ONE ' ');
12 L._ISTREAM := open_in (s^".pl");
13 advance();
14 prog := initprog::clauses(); close_in (!L._ISTREAM))先頭につけることで読み込めるようにす
   (*る
15 *)
16 | _ -> error())
17 | _ -> let t = terms() in
18 (check(L.ONE ' '); let _ = E.eval(!prog, t) in ())
19
20 and term() = match !tok with
21 L.ONE '(' -> (eat (L.ONE '('); let t = term() in eat(L.ONE ')');t)
22 |L.VID s -> (eat (L.VID ""); eat (L.IS); let a = arithmexp() in E.App("is",[E.
   Var
23 s;a]))
24 | _ -> (let p = predicate() in p)
25
26 and terms() = (let t = term() in let t' = terms'() in t::t')
27
28 and terms'() = match !tok with
29 L.ONE ', ' -> (eat (L.ONE ', '); let t = term() in let t' = terms'() in t::t')
30 | _ -> []
31
32 and predicate() = match !tok with
33 L.CID s -> (eat (L.CID ""); eat (L.ONE '('); let a = args() in eat (L.ONE ')');
   E.App
34 (s, a))
35 | _ -> error()
36
37 and args() = (let a = arithmexp() in let a' = args'() in a::a')
38
39 and args'() = match !tok with
40 L.ONE ', ' -> (eat (L.ONE ', '); let a = arithmexp() in let a' = args'() in a::a
   ')
```

```

41 | _ -> []
42
43 and arithmexp () = arithmexp'(arithmterm())
44
45 and arithmexp' e = match !tok with
46 L.ONE '+' -> (eat (L.ONE '+'); arithmexp'(E.App("+", [e;arithmterm()])))
47 | L.ONE '-' -> (eat (L.ONE '-'); arithmexp'(E.App("-", [e;arithmterm()])))
48 | _ -> e
49
50 and tail_opt s= match !tok with
51 L.ONE '(' -> (eat (L.ONE '('); let a = args() in eat (L.ONE ')'); E.App(s, a))
52 | _ -> (E.Atom s)
53
54 and arithmterm() = arithmterm'(arithmfactor())
55
56 and arithmterm' t = match !tok with
57 | L.ONE '*' -> (eat (L.ONE '*'); arithmterm'(E.App("*", [t;arithmfactor()])))
58 | L.ONE '/' -> (eat (L.ONE '/'); arithmterm'(E.App("/", [t;arithmfactor()])))
59 | _ -> t数値であれば数値をそのまま返す(**)
60
61 and list() = match !tok with
62 L.ONE ']' -> (E.Atom "nil")
63 | _ -> (let a = arithmexp() in let lo = list_opt() in E.App ("cons", [a; lo]))
64
65 and list_opt() = match !tok with
66 L.ONE '|' -> (eat (L.ONE '|'); let i = arithmfactor() in i)
67 | L.ONE ',' -> (eat (L.ONE ','); let l = list() in l)
68 | _ -> (E.Atom "nil")
69
70 and arithmfactor() = match !tok with
71 L.ONE '(' -> (eat (L.ONE '('); let a = arithmexp() in eat (L.ONE ')'); a)
72 | L.ONE '-' -> (eat (L.ONE '-'); let a = arithmexp() in E.App("!", [a]))
73 | L.ONE '[' -> (eat (L.ONE '['); let l = list() in eat (L.ONE ']'); l)
74 | L.CID s -> (eat (L.CID ""); tail_opt s)
75 | L.VID s -> (eat (L.VID ""); E.Var s)
76 | L.NUM n -> (eat (L.NUM ""); E.Atom n)
77 | _ -> error()
78
79
80
81
82 end

```

14 実行結果

```
# run();;
?-open isono.

?- grandfather(namihei,X).
X=tara
Yes.
No.

?- X is 5.
X=5
Yes.
No.

?- X is 4.
X=4
Yes.
No.

?- X is 1+2-3*4/5.
X=-(/(5,*(4,3)),+(2,1))
Yes.
No.

?-
```

図 7 実行結果

問題 8: 「VID IS arithmexp」の構文で,arithmexp を計算した結果を変数に代入するように拡張しなさい.

15 ソースコードの変更箇所

```
1 問題で追加行目
2 (*8,10*)
3 exception Not_number
4 exception Compiler_error
5 exception Continue行目
6 (*136*)
7 let rec solve (program, question, l, result, depth) = match question with
8 [] -> succeed result l
9 | goal::goals ->
10 let onestep _ clause =
11 match List.map (rename (string_of_int depth)) clause with
12 [] -> raise Compiler_error
13 | head::conds ->
14 let (unifiable, unifier) = mgu(cal head,cal goal) in
15 if unifiable then
16 solve (program, List.map unifier (conds@goals), l ,List.map unifier result,
17       depth+1)
18 else true
19 in List.fold_left onestep true program行目
20 (*539*)
21 let rec run() =
22 print_string "?- ";
23 while true do
24 flush stdout; Lexer._ISTREAM := stdin;
25 (try
26 Parser.advance(); Parser.command() with Evaluator.Continue ->()
27 | Evaluator.Not_number -> print_string "No_number");print_string "\n?-"
28
29 done
```

16 実行結果

```
val run : unit -> unit = <fun>
# run();;
?- open isono.

?-grandfather(namihei,X).
X=tara
Yes.
No.

?-X is 1+2.
X=3
Yes.
No.

?-Y is 1+X.
No_number
?-X is 1+2*4.
X=9
Yes.
No.

?-
```

図 8 実行結果